Mapa do código, Trabalho 2

Clínica Vet, versão `b1228b3` da branch `main`. Cada pergunta traz o arquivo e a linha onde a coisa acontece e o que dizer em voz alta. Se o código mudar, as linhas se deslocam; o nome do método continua valendo.

Texto para ler Perguntas do professor Editar, por entidade Fluxos Camadas Pontos fracos

Texto para ler e repetir

Leia em voz alta, de cima para baixo. A ordem é a que a apresentação costuma seguir.

1. O que mudou do Trabalho 1 para o Trabalho 2

O Trabalho 1 já tinha MVC, login por sessão e CRUDs, mas os models misturavam duas tarefas, representar o dado e falar com o banco, e os controllers carregavam muita validação. No Trabalho 2 separamos isso em camadas com uma responsabilidade cada. As entidades (`app/Entities`) representam os objetos. Os repositórios (`app/Repositories`) concentram todo o SQL. Os validadores (`app/Validators`) concentram as regras de formulário. O controller só coordena.

Além disso, o Router passou a entender parâmetros na URL, o projeto ganhou um pacote Composer de verdade, o Monolog, e os erros inesperados passaram a ser registrados em arquivo.

2. O caminho de uma requisição

Todo acesso entra por `public/index.php` . Ele carrega o autoload do Composer, inicia a sessão, configura o Monolog e os handlers de erro, registra as rotas e chama `dispatch` .

O Router compara o método e o caminho com as rotas registradas. Quando o caminho tem `{id}` , esse trecho vira um grupo de expressão regular e o valor capturado é passado como argumento da action. Por isso `/tutores/editar/5` chama `TutorController::editar('5')` . Se nenhuma rota casa, a resposta é 404.

O controller começa protegendo a página com `Auth::requireLogin()` , busca os dados no repositório e chama `render` , que transforma o array em variáveis e

dados no repositório e chama `render`, que transforma o array em variáveis e carrega a view. A view só imprime HTML, sempre com `htmlspecialchars` nos valores.

3. Como um formulário é salvo

No POST, o controller limpa os campos com `Validator::limpar` e entrega os dados ao validador da entidade. O validador usa as regras do `Validator` base (obrigatório, CPF, telefone, CEP, UF, data, lista permitida). Quando a regra depende do banco, ele pergunta ao repositório: `cpfExists` para CPF duplicado, `existeConflito` para dois atendimentos do mesmo veterinário no mesmo horário.

Se há erros, eles ficam na sessão junto com o que foi digitado, e o usuário volta ao formulário com cada campo marcado. Se está tudo certo, `Entity::fromArray` monta o objeto, o repositório executa o INSERT ou o UPDATE com prepared statement, o controller grava a mensagem de sucesso e redireciona. Esse é o padrão Post/Redirect/Get: apertar F5 não reenvia o formulário.

4. Banco de dados

A conexão está em `Database::getConnection()`: um único PDO, com exceções ativadas, `FETCH_ASSOC` e `EMULATE_PREPARES` desligado. `Repository` é uma classe abstrata que guarda o PDO, o nome da tabela e o `find`. Cada repositório filho declara a tabela, implementa `paraEntidade` e escreve as consultas próprias. Nenhum valor vindo do usuário entra no SQL por concatenação, tudo vai por parâmetro.

5. Autenticação e perfis

No login, o `AuthController` busca o usuário por e-mail, confere a senha com `password_verify` e exige que o usuário esteja ativo. `Auth::login` regenera o id da sessão e guarda id, nome e perfil. As senhas são gravadas com `password_hash`. `requireLogin` redireciona para `/login` e `requireRole('admin')` mostra a página 403. Hoje só a área de usuários exige admin; nas demais telas basta estar logado.

6. Erros, log e Composer

`set_error_handler` transforma avisos do PHP em exceção. `set_exception_handler` registra qualquer exceção não tratada com o Monolog em `storage/logs/app.log`, responde HTTP 500 e mostra uma página amigável, sem stack trace. O Monolog entra pelo Composer e o `composer.lock` trava as versões. O autoload PSR-4 mapeia `App\` para `app/`, então nenhuma classe precisa de `require` manual.

`require` manual.

7. Por que decidimos assim

- **Repositório em vez de model:** separa persistência de representação. O SQL fica em um lugar só.
- **Entidade em vez de array:** propriedades tipadas, e o comportamento fica junto do dado (`cpfFormatado` , `idade` , `estaCancelada`), com menos chave de array escrita à mão.
- **Validador separado:** o controller fica curto e cada validador recebe o repositório de que precisa pelo construtor.
- **Router próprio:** o enunciado proíbe framework e precisávamos de URL transparente. Regex com grupos nomeados resolve com pouco código.
- **Monolog:** é a biblioteca de log mais usada no ecossistema PHP e não é framework.

8. Quem fez o quê

O Andrey ficou com a base: Router com parâmetros, Database, Controller base, Auth, a área de usuários, o Monolog e a documentação de instalação. O Davi ficou com o domínio: as regras do Validator, a classe base Repository, as entidades, os repositórios e os validadores de tutor, animal e consulta, e a URL transparente nessas rotas.

9. O que dizer se apontarem uma limitação

Diga com calma e já aponte o próximo passo. A validação de usuário ainda está no controller, e um `UsuarioValidator` seria o passo natural. Só a área de usuários tem restrição por perfil. A sessão guarda poucos dados e o `Auth::user()` devolve um array. Mais detalhes na seção de pontos fracos, mais abaixo.

Procurar: router, CPF, sessão, PDO, erro…

Todas Rotas Banco OOP MVC Autenticação Validação Erros e log Composer

39 de 39 perguntas na tela · 0 marcadas como “já sei”

Abrir todas | Fechar todas | Zerar “já sei”

Perguntas do professor

Leia a pergunta, responda em voz alta e só então abra. Marque “já sei” para acompanhar o progresso.

+ ROTAS **Onde as rotas são registradas?**

+ ROTAS **Como funciona a URL transparente /tutores/editar/5?**

+ ROTAS **Onde é gerado o erro 404?**

+ ROTAS **Por que existe get() e post() separados?**

+ ROTAS **A rota / existe?**

+ BANCO **Onde a conexão com o banco é criada?**

+ BANCO **Onde está o SQL do sistema?**

+ BANCO **Como vocês evitam SQL injection?**

+ BANCO **O que a classe Repository faz e o que os filhos fazem?**

+ BANCO **Onde uma linha do banco vira objeto?**

+ BANCO **Onde estão as tabelas e o usuário de teste?**

+ OOP Onde aparecem herança, classe abstrata e injeção de dependência?

+ OOP O que é uma Entity e por que não usar array?

+ OOP Tem regra de negócio dentro das entidades?

+ OOP Qual a diferença entre Entity, Repository e Validator?

+ MVC Como o MVC está separado?

+ MVC Como o controller entrega dados para a view?

+ MVC Já teve algum bug estranho de variável na view?

+ MVC O que é o padrão Post/Redirect/Get que vocês usam?

+ MVC Como aparecem as mensagens de feedback?

+ MVC Como funcionam os filtros da listagem de consultas?

+ AUTENTICAÇÃO O que acontece quando alguém faz login?

+ AUTENTICAÇÃO Como a senha é guardada?

+ AUTENTICAÇÃO Como uma página é protegida?

+ AUTENTICAÇÃO Como é o logout?

+ AUTENTICAÇAO O que acontece com um usuário inativo?

+ AUTENTICAÇÃO Quem pode ver o quê?

+ VALIDAÇÃO Onde a validação acontece?

+ VALIDAÇÃO Como o CPF é validado?

+ VALIDAÇÃO Como impedem CPF duplicado?

+ VALIDAÇÃO Como impedem dois atendimentos no mesmo horário?

+ VALIDAÇÃO Consulta no passado é aceita?

+ VALIDAÇÃO Para que serve Validator::limpar?

+ VALIDAÇÃO Como o usuário vê qual campo errou?

+ ERROS E LOG O que acontece quando dá um erro inesperado?

+ ERROS E LOG Onde está o Monolog e onde o log é gravado?

+ COMPOSER Como funciona o autoload e para que serve o Composer aqui?

+ COMPOSER Qual pacote vocês usam e por quê?

+ COMPOSER Por que não usaram um framework?

Editar um registro: que arquivo faz o quê

Cliente, no sistema, é o **tutor**. As quatro telas seguem o mesmo caminho; as diferenças estão marcadas.

	USUÁRIO	TUTOR
Rotas public/index.php	:49 GET :50 POST	:56 GET :57 POST
Abrir o formulário GET, no controller	UsuarioController::edit :58	TutorController::editar :39
Proteção da página	requireRole('admin')	requireLogin() na prim
Buscar o registro todos usam Repository::find, Repository.php:27	direto em edit :58 ($this→usuarios->find)	buscar :87
Montar a tela render em Controller.php:7	Views/usuarios/form.php	Views/tutores/form.php formulario :51
Receber o POST no controller	UsuarioController::update :77	atualizar :45 → salvar :61
Validar	UsuarioController::validar :112 única que ficou dentro do	TutorValidator::validar :18 CPF duplicado:

	controller	TutorRepository::cpfExists :24
Mostrar erro	mensagem única em flash, mais `old` na sessão	erros por campo em `$_S` `is-invalid`
Montar o objeto	`new Usuario(...)` dentro de **`update`**	**`Tutor::fromArray :27`**
UPDATE no banco prepared statement	**`UsuarioRepository::update :68`**	**`TutorRepository::update :49`**
Sucesso flash + redirect, Controller.php:25	volta para `/usuarios`	volta para `/tutores`

Fluxos para saber de cabeça

O professor costuma pedir “o que acontece quando eu clico em X”. Siga a ordem.

Login

`public/index.php:41`	GET / ou /login chega no Router, que chama AuthController::showLogin.
`Views/auth/login.php`	Exibe o formulário (e o e-mail digitado, se a tentativa anterior falhou).
`AuthController.php:21`	POST /login: confere se os campos vieram preenchidos.
`UsuarioRepository.php:27`	findByEmail devolve uma Entity Usuario ou null.
`AuthController.php:21-41`	password_verify e checagem de ativo. Falhou:

	flash de erro e volta ao login.
`Auth.php:9`	Deu certo: regenera o id da sessão e grava id, nome e perfil.
`AuthController.php:50`	paginaInicial redireciona: admin para /usuarios, os demais para /consultas.

Editar um tutor (GET e POST)

`Router.php:44-53`	GET /tutores/editar/5: o regex captura id = 5 e chama TutorController::editar('5').
`TutorController.php:39`	requireLogin, depois buscar((int) $id) usa o repositório; não achou: flash e volta à lista.
`Repository.php:27`	find faz o SELECT preparado e devolve a linha convertida em Tutor.
`Controller.php:7`	render('tutores/form', ...) extrai as variáveis e carrega a view.
`TutorController.php:61`	POST /tutores/editar/5: salvar limpa os dados com Validator::limpar.
`TutorValidator.php:18`	Valida os campos e checa CPF duplicado via TutorRepository::cpfExists.
`TutorController.php:65-69`	Com erro: grava erros e old na sessão e redireciona ao formulário.
`TutorRepository.php:49`	Sem erro: Tutor::fromArray monta o objeto e update executa o UPDATE preparado.
`Controller.php:25`	flash de sucesso e redirect para /tutores; o header mostra o alerta.

Agendar uma consulta

`ConsultaController.php:42`	GET /consultas/novo: carrega o formulário com animais e veterinários ativos.
`ConsultaRepository.php:71`	veterinarios() lista só perfil veterinário e ativo.
`ConsultaController.php:78`	POST: salvar limpa os dados e chama o ConsultaValidator.
`ConsultaValidator.php:23`	Valida animal, veterinário, data, hora e status; confere se animal e veterinário existem.
`ConsultaValidator.php:46-56`	Agendada não pode ser no passado; a data precisa existir de verdade.
`ConsultaRepository.php:52`	existeConflito: mesmo veterinário e horário, ignorando canceladas e a própria consulta.
`ConsultaRepository.php:79`	create insere no banco; flash de sucesso e redirect para /consultas.

Cada camada em uma frase

> Router conecta a URL ao controller; o controller coordena; o validator verifica; a entity representa; o repository persiste; a view apresenta; o Database fornece o PDO.

Router

Transforma a URL e o método HTTP em controller + action, extraindo parâmetros como {id}.

`app/Core/Router.php`

Controller

Recebe a requisição, protege com Auth, chama validator e repositório, escolhe render ou redirect.

`app/Controllers/*`

Validator

Regras de formulário no servidor; devolve

Entity

Objeto de domínio tipado, com fromArray,

erros por campo.

`app/Core/Validator.php`, `app/Validators/*`

toArray e pequenos comportamentos.

`app/Entities/*`

Repository

Único lugar com SQL; usa PDO com prepared statements e devolve entidades.

`app/Repositories/*`, `app/Core/Repository.php`

View

Só HTML e echo com htmlspecialchars; sem consulta e sem regra de negócio.

`app/Views/*`

Database

Cria e reaproveita a conexão PDO a partir do config.php.

`app/Core/Database.php`

Auth

Sessão: login, logout, requireLogin e requireRole.

`app/Core/Auth.php`

Pontos que ele pode cutucar

São coisas reais do código. Melhor você falar antes do que ser pego.

A validação de usuário ficou dentro do controller

Tutor, animal e consulta têm classe validadora própria; usuário ainda usa um método privado `validar` no controller e junta as mensagens em um único flash. Se